SHETH L.U.J & SIR MV COLLEGE

Aim:KNN

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load the Iris dataset
df = pd.read_csv('Iris.csv')

# Select input features and target variable
X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
y = df['Species']

# Split the dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Standardize the feature values
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create the K-NN model with K = 5
model = KNeighborsClassifier(n_neighbors=5)

# Train the K-NN model
model.fit(X_train, y_train)
```


Purvi.R.Karkera
T086

```
# Predict the classes for test data
y_pred = model.predict(X_test)

# Calculate accuracy and error
accuracy = accuracy_score(y_test, y_pred)
error = 1 - accuracy

print("K-NN Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")
print("Prediction Error:", error)
print("Error Percentage:", error * 100, "%")

# Display the confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Display the classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Visualize the confusion matrix
plt.figure(figsize=(6, 5))

plt.imshow(cm)
plt.title("K-NN Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.colorbar()

classes = model.classes_

plt.xticks(range(len(classes)), classes, rotation=20)
plt.yticks(range(len(classes)), classes)

# Display values inside the confusion matrix
for i in range(len(classes)):
    for j in range(len(classes)):
        plt.text(j, i, cm[i, j], ha='center', va='center')

plt.tight_layout()
plt.show()
```

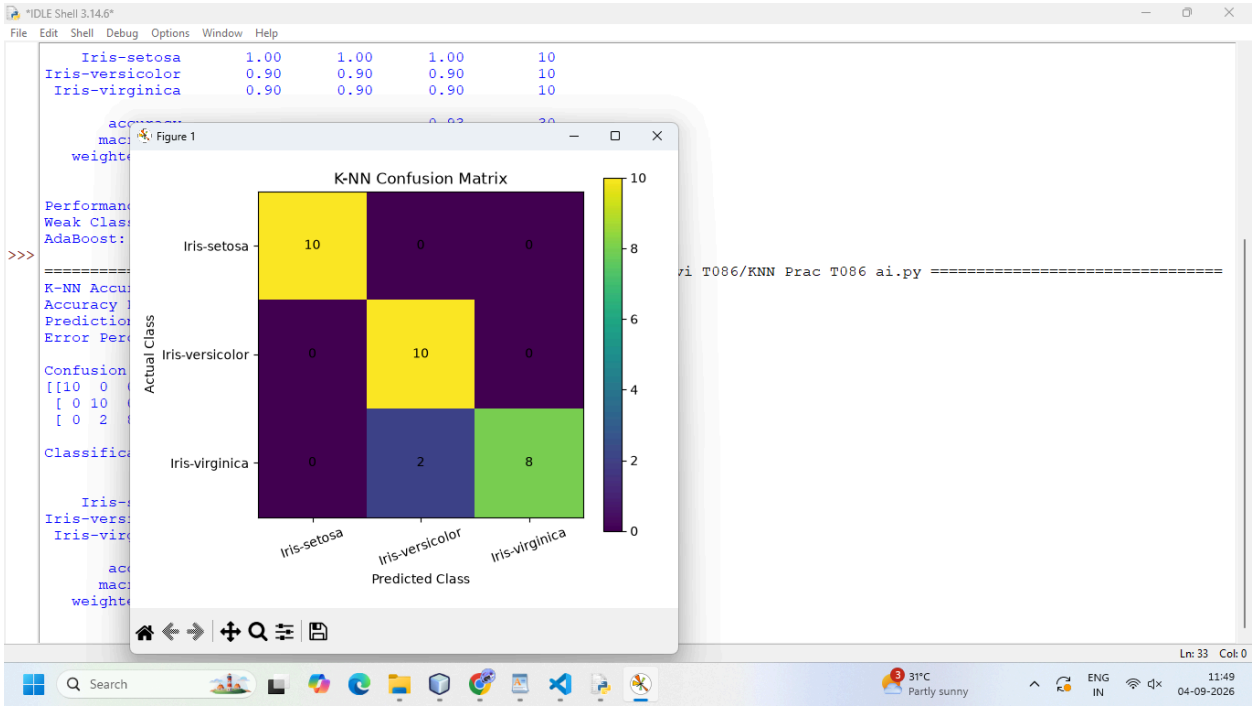

Purvi.R.Karkera
T086

```
# Use two features for visualizing the K-NN decision boundary
X_visual = df[['PetalLengthCm', 'PetalWidthCm']]
y_visual = df['Species']

# Split the visualization features into training and testing data
Xv_train, Xv_test, yv_train, yv_test = train_test_split(
    X_visual,
    y_visual,
    test_size=0.2,
    random_state=42,
    stratify=y_visual
)

# Standardize the visualization features
visual_scaler = StandardScaler()

Xv_train = visual_scaler.fit_transform(Xv_train)
Xv_test = visual_scaler.transform(Xv_test)

# Train K-NN using the two visualization features
visual_model = KNeighborsClassifier(n_neighbors=5)
visual_model.fit(Xv_train, yv_train)

# Create a mesh for the decision boundary
x_min = Xv_train[:, 0].min() - 1
x_max = Xv_train[:, 0].max() + 1
y_min = Xv_train[:, 1].min() - 1
y_max = Xv_train[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.arange(x_min, x_max, 0.02),
    np.arange(y_min, y_max, 0.02)
)

# Predict the class for every point in the mesh
Z = visual_model.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

# Convert class labels into numeric values for plotting
class_to_number = {
    classes[0]: 0,
    classes[1]: 1,
    classes[2]: 2

Purvi.R.Karkera
T086
```

```
}

Z = np.array([class_to_number[value] for value in Z])
Z = Z.reshape(xx.shape)

# Plot the K-NN decision boundary
plt.figure(figsize=(9, 6))

plt.contourf(xx, yy, Z, alpha=0.3)

# Convert training labels into numeric values
train_numbers = np.array([
    class_to_number[value] for value in yv_train
])

plt.scatter(
    Xv_train[:, 0],
    Xv_train[:, 1],
    c=train_numbers,
    edgecolors='black',
    label='Training Data'
)

plt.xlabel('Standardized Petal Length')
plt.ylabel('Standardized Petal Width')
plt.title('K-NN Decision Boundary (K = 5)')
plt.legend()

plt.show()
```

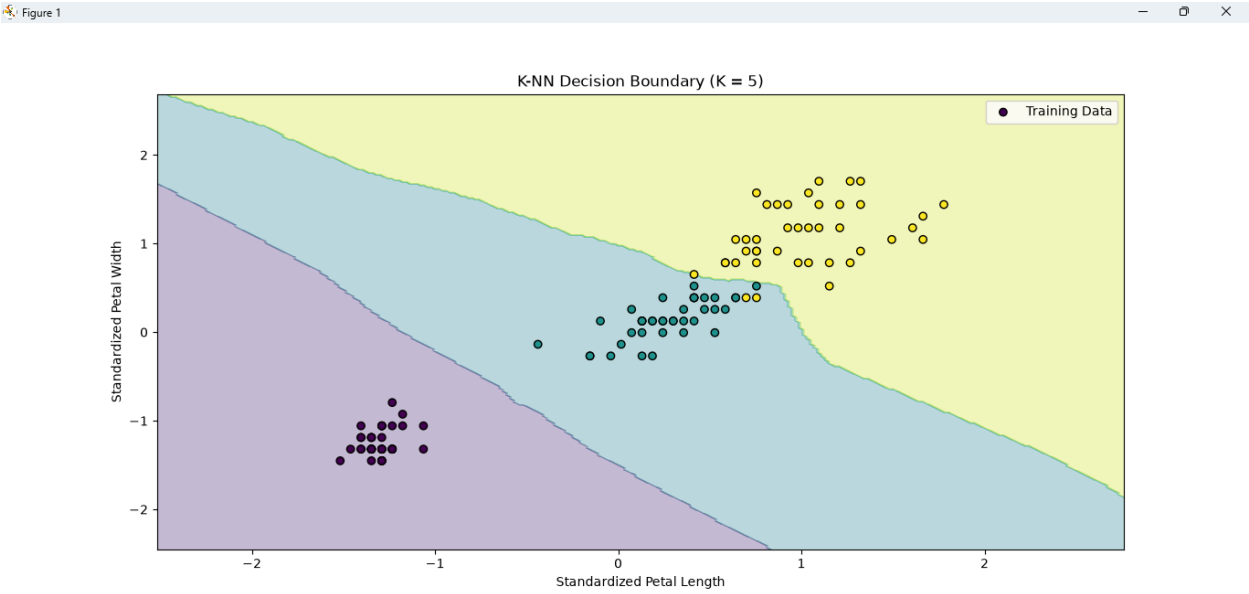
SHETH L.U.J & SIR MV COLLEGE

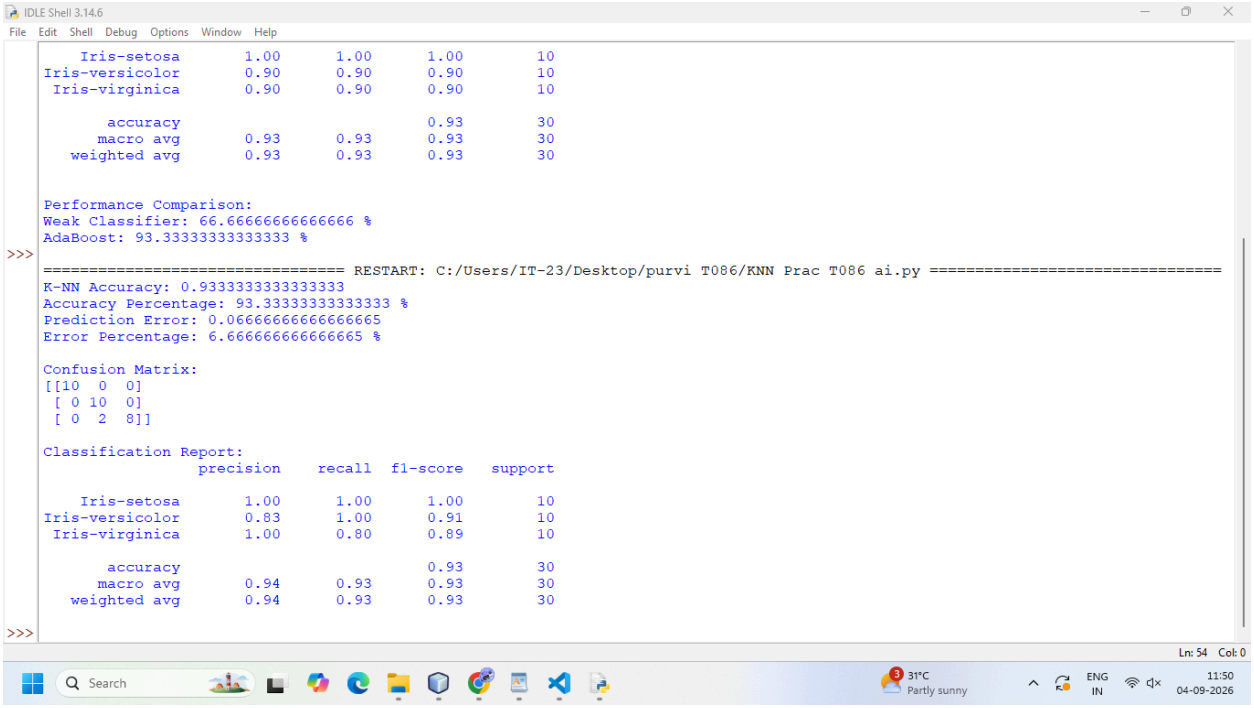
Output:



Purvi.R.Karkera
T086

SHETH L.U.J & SIR MV COLLEGE





Purvi.R.Karkera
T086